

Week 2

1. DFS code for 8 puzzle

```
def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()

def depth_limited_search(state, goal, path, limit):
    if state == goal:
        return path

    if len(path) - 1 == limit:
        return None

    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_zero = new_row * 3 + new_col

            new_state = list(state)

            new_state[zero], new_state[new_zero] = (
                new_state[new_zero],
                new_state[zero]
            )
```

```

        new_state = tuple(new_state)

        if new_state not in path:

            result = depth_limited_search(
                new_state,
                goal,
                path + [new_state],
                limit
            )

            if result:
                return result

    return None

```

```

def ids(start, goal):
    for depth in range(30):

        result = depth_limited_search(
            start,
            goal,
            [start],
            depth
        )

        if result:
            return result, depth

    return None, None

```

```

start = (
    5, 4, 6,
    1, 8, 0,
    7, 3, 2
)

```

```

goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

```

)

```
print("Initial State:")  
print_puzzle(start)
```

```
solution, depth = ids(start, goal)
```

```
if solution:  
    print("IDS Solution Found at Depth:", depth)  
    print()
```

```
    for i, state in enumerate(solution):  
        print("Step", i)  
        print_puzzle(state)
```

```
else:  
    print("No solution found")
```

File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>

===== RESTART: C:/Users/Student/Desktop/NITYA/DFS.py =====

Initial State:

5 4 6

1 8 0

7 3 2

IDS Solution Found at Depth: 19

Step 0

5 4 6

1 8 0

7 3 2

Step 1

5 4 6

1 8 2

7 3 0

Step 2

5 4 6

1 8 2

7 0 3

Step 3

5 4 6

1 0 2

7 8 3

Step 4

5 0 6

1 4 2

7 8 3

Step 5

0 5 6

1 4 2

7 8 3

Step 6

1 5 6

0 4 2

7 8 3

Step 7

1 5 6

4 0 2

7 8 3

Step 8

1 0 6

4 5 2

7 8 3

Step 9

1 6 0

4 5 2

7 8 3

Step 9
1 6 0
4 5 2
7 8 3

Step 10
1 6 2
4 5 0
7 8 3

Step 11
1 6 2
4 5 3
7 8 0

Step 12
1 6 2
4 5 3
7 0 8

Step 13
1 6 2
4 0 3
7 5 8

Step 14
1 0 2
4 6 3
7 5 8

Step 15
1 2 0
4 6 3
7 5 8

Step 16
1 2 3
4 6 0
7 5 8

Step 17
1 2 3
4 0 6
7 5 8

Step 18
1 2 3
4 5 6
7 0 8

Step 19
1 2 3
4 5 6
7 8 0

2. IDS code for 8 puzzle

```
def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()

def depth_limited_search(state, goal, path, limit):

    if state == goal:
        return path

    if len(path) - 1 == limit:
        return None

    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    for dr, dc in moves:

        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```

new_zero = new_row * 3 + new_col

    new_state = list(state)

        new_state[zero], new_state[new_zero] = (
            new_state[new_zero],
            new_state[zero]
        )

    new_state = tuple(new_state)

    if new_state not in path:

        result = depth_limited_search(
            new_state,
            goal,
            path + [new_state],
            limit
        )

        if result:
            return result

return None

```

```

def ids(start, goal):

    for depth in range(30):

        result = depth_limited_search(
            start,
            goal,
            [start],
            depth

```

```

    )

    if result:
        return result, depth

    return None, None

start = (
    5, 4, 6,
    1, 8, 0,
    7, 3, 2
)
goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)
print("Initial State:")
print_puzzle(start)
solution, depth = ids(start, goal)
if solution:

    print("IDS Solution Found at Depth:", depth)
    print()

    for i, state in enumerate(solution):

        print("Step", i)
        print_puzzle(state)

else:

    print("No solution found")

```

```
>>> ===== RESTART: C:/Users/Student/Desktop/NITYA/IDS.py =====
Initial State:
5 4 6
1 8 0
7 3 2

IDS Solution Found at Depth: 19

Step 0
5 4 6
1 8 0
7 3 2

Step 1
5 4 6
1 8 2
7 3 0

Step 2
5 4 6
1 8 2
7 0 3

Step 3
5 4 6
1 0 2
7 8 3

Step 4
5 0 6
1 4 2
7 8 3

Step 5
0 5 6
1 4 2
7 8 3

Step 6
1 5 6
0 4 2
7 8 3

Step 7
1 5 6
4 0 2
7 8 3

Step 8
1 0 6
4 5 2
7 8 3

Step 9
1 6 0
4 5 2
7 8 3
```

1 0 6
4 5 2
7 8 3

Step 9
1 6 0
4 5 2
7 8 3

Step 10
1 6 2
4 5 0
7 8 3

Step 11
1 6 2
4 5 3
7 8 0

Step 12
1 6 2
4 5 3
7 0 8

Step 13
1 6 2
4 0 3
7 5 8

Step 14
1 0 2
4 6 3
7 5 8

Step 15
1 2 0
4 6 3
7 5 8

Step 16
1 2 3
4 6 0
7 5 8

Step 17
1 2 3
4 0 6
7 5 8

Step 18
1 2 3
4 5 6
7 0 8

Step 19
1 2 3
4 5 6
7 8 0